University of Jordan
Department of Computer Engineering
Digital Logic Laboratory

Pizza Delivery System

Lab Project Report

Submitted by

Sara Samer Alswealmeen

Instructor: Saadeh Sweidan
Semester: Fall 2024

Table of Contents

1. Introduction **4**
2. Top-Level Overview **5**
2.1 System Requirements 5
2.2 System Flowcharts 5
2.3 Overall Design 6
3. Description of Design Parts **7**
3.1 Before Taking in Orders 7
3.2 Taking in Orders 8
3.3 Orders Processing 10
3.3.1 Four-Bit Counter 10
3.3.2 LED Device 12
3.3.3 Successful Deliveries Counter 14
3.3.4 Display Digits 15
4. Project Functional Simulation **18**
4.1 Simulation Case 1 18
4.2 Simulation Case 2 19
4.3 Simulation Case 3 19
4.4 Simulation Case 4 20
5. Conclusion **21**

List of Figures, Tables and Listings

Figures

Figure 1.1: Block diagram of the pizza delivery system 4
Figure 2.1: Flowchart of the system (1): response to the switches 5
Figure 2.2: Flowchart of the system (2): outcome of a delivery 6
Figure 2.3: Overall view of the design 6
Figure 3.1: Block diagram of the input processing 8
Figure 3.2: Connection of the devices that process the switch inputs 9
Figure 3.3: Block diagram of a single output bit of the 4-bit counter 11
Figure 3.4: Connection of the 4-bit counter to the hardware that controls its restart 12
Figure 3.5: Block design of the device that processes the delivery results 13
Figure 3.6: Processing of the Success switch to save its value in each round 14
Figure 3.7: Block diagram of the 3-bit counter connections 14
Figure 3.8: Display Digit 1 connected to its 7-segment decoder 16
Figure 3.9: Display Digit 0 connected to its 7-segment decoder through a multiplexer 17
Figure 4.1: Simulation waveform of case 1 18
Figure 4.2: Simulation waveform of case 2 19
Figure 4.3: Simulation waveform of case 3 19
Figure 4.4: Simulation waveform of case 4 20

Tables

Table 3.1: 3-bit counter states 7
Table 3.2: 4-bit counter states 10

Listings

Listing 3.1: Verilog HDL implementation of the 3-bit counter 7
Listing 3.2: Verilog HDL implementation of the input processing 8
Listing 3.3: Verilog HDL implementation of the order-turn check 9
Listing 3.4: Verilog HDL implementation of the 4-bit counter 11
Listing 3.5: Verilog HDL implementation of the LED device 13
Listing 3.6: Verilog HDL implementation of the 7-segment decoder 15
Listing 3.7: Verilog HDL implementation of the display multiplexer 17

1. Introduction

This report describes the design and implementation of a pizza delivery system that handles up to five delivery orders in sequence and reports the result of each delivery using LEDs and 7-segment display digits (Figure 1.1).

The objective of the project is to apply the combinational and sequential building blocks studied in the Digital Logic Laboratory, including logic gates, flip-flops, multiplexers, demultiplexers and decoders, to build a complete working system. The design is described structurally in Verilog HDL, and its outputs are shown on the LEDs and 7-segment displays of the laboratory kit.

The rest of the report is organized as follows. Section 2 lists the system requirements and presents the flowcharts and the overall design. Section 3 describes each design component in detail, together with its Verilog implementation. Section 4 presents the functional simulation results for four test cases, and Section 5 concludes the report.

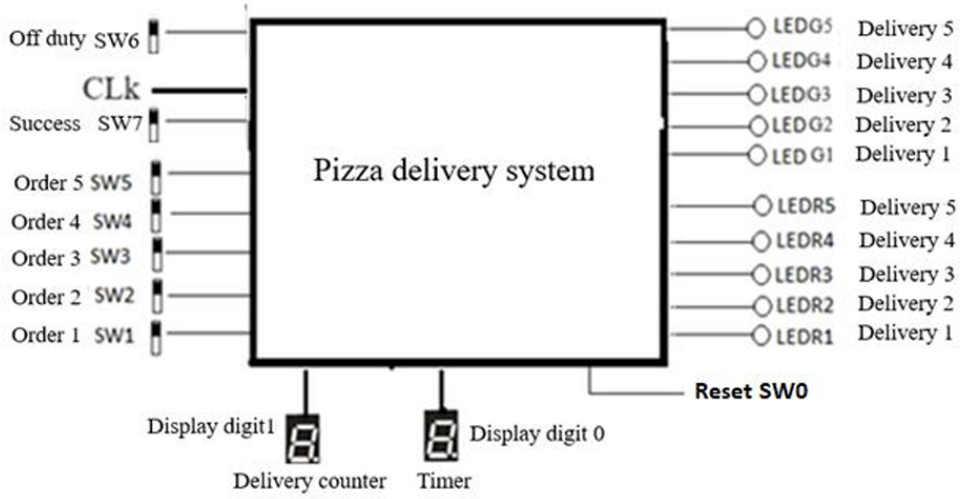


figure 1. Pizza delivery system

Figure 1.1: Block diagram of the pizza delivery system

2. Top-Level Overview

2.1 System Requirements

Before presenting the implementation, the key requirements that guided the design process are listed below. Each part of the design was checked against them.

- While SW0 is ON, the system is OFF: all LEDs are OFF and both display digits show 0.
- When the system is ON and no order switch has been activated yet, Display Digit 0 counts from 0 to 5.
- When an order switch is activated in its correct turn, its green LED turns ON and Display Digit 0 counts down from 8 to 0. If the Success switch is turned ON during this time, the green LED stays ON and the red LED stays OFF; otherwise, the green LED turns OFF and the red LED turns ON.
- Display Digit 1 shows the number of successful deliveries.
- If the Off Duty switch is ON, the current order is completed but no new order is accepted.

2.2 System Flowcharts

The overall behavior of the system is summarized by the flowcharts in Figures 2.1 and 2.2. Figure 2.1 shows how the system responds to the switches, and Figure 2.2 shows how the outcome of each delivery is determined once the countdown of an order reaches zero.

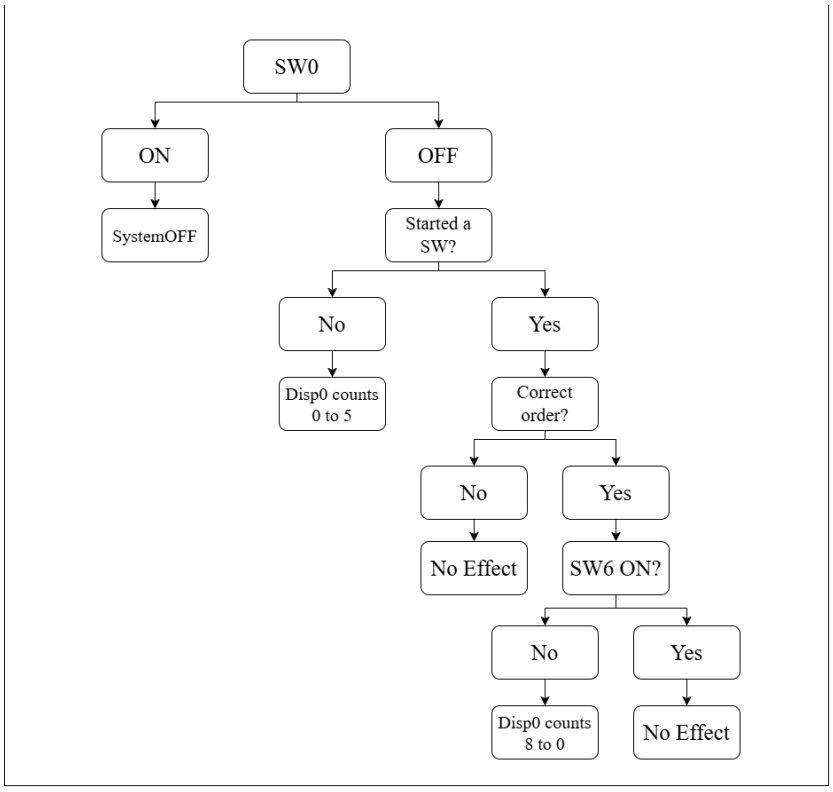


Figure 2.1: Flowchart of the system (1): response to the switches

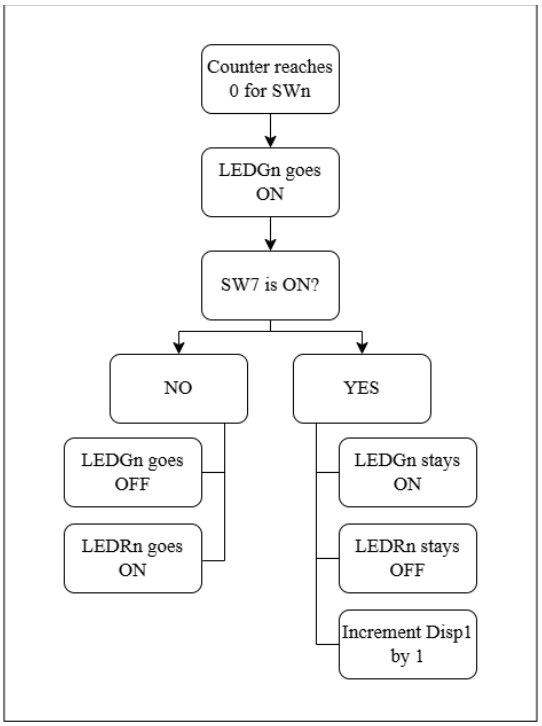


Figure 2.2: Flowchart of the system (2): outcome of a delivery

2.3 Overall Design

Figure 2.3 shows the overall look of the complete design. The following section discusses each of its components in turn.

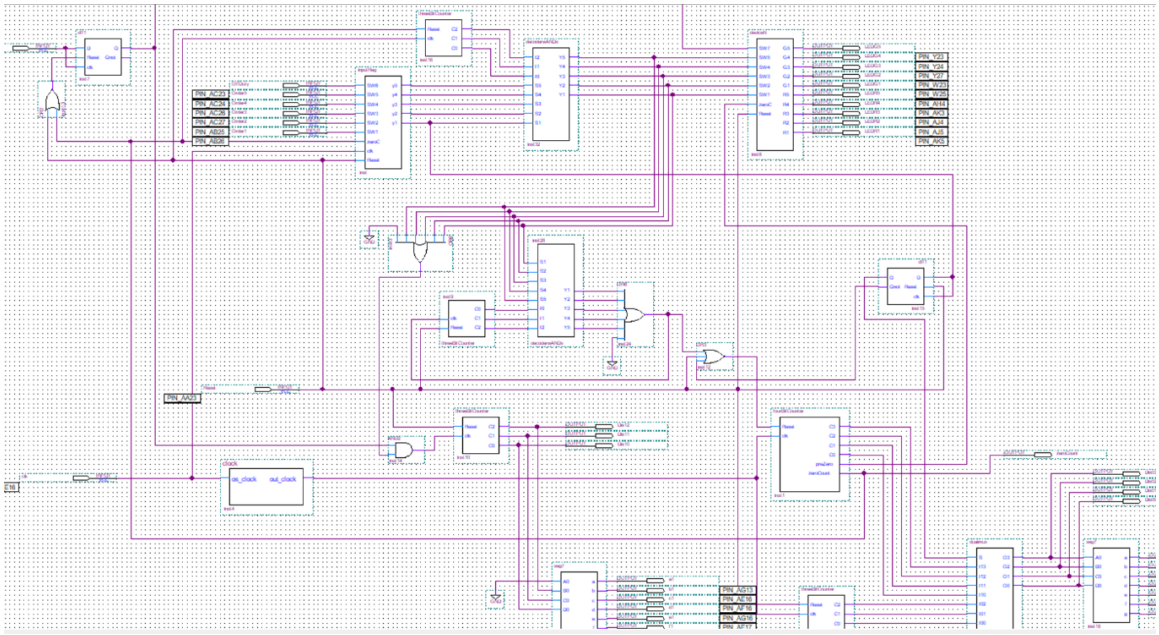


Figure 2.3: Overall view of the design

3. Description of Design Parts

3.1 Before Taking in Orders

After the system is started and before any order is taken, Display Digit 0 counts from 0 to 5. This is implemented with a 3-bit counter register (D2 D1 D0) that follows the state table in Table 3.1; once it reaches 5 (101) it stays there.

Table 3.1: 3-bit counter states

Current State (C2 C1 C0)	Next State (D2 D1 D0)
000	001
001	010
010	011
011	100
100	101
101	101

From this table, the equation for each bit of the register is:

```
D0 = C2 + (~C0)
D1 = (~C2)·(~C1)·C0 + C1·(~C0)
D2 = C1·C0 + C2
```

The counter was implemented structurally in Verilog HDL, as shown in Listing 3.1.

```
module threeBitCounter (Reset, clk, C2, C1, C0);
input Reset, clk;
output C2, C1, C0;

Invgate (n0, C0);
Invgate (n1, C1);
Invgate (n2, C2);

Orgate2 (D0, n0, C2);//D0 = C2+nC0

Andgate3 (w1, n2, n1, C0);
Andgate2 (w2, C1, n0);
Orgate2 (D1, w1, w2);//D1 = nC2.nC1.C0 + C1.nC0

Andgate2 (w3, C1, C0);
Orgate2 (D2, w3, C2);// D2 = C1.C0 + C2

//module dff1(D,Reset,clk,Q,Qnot);
dff1 (D0, Reset, clk, C0, nC0);
dff1 (D1, Reset, clk, C1, nC1);
dff1 (D2, Reset, clk, C2, nC2);

endmodule
```

Listing 3.1: Verilog HDL implementation of the 3-bit counter

3.2 Taking in Orders

An order can only be accepted when the Off Duty switch is OFF. To guarantee this, and to make sure that an order accepted before the Off Duty switch was turned ON is kept, the values of the order switches are stored in D flip-flops whose clock is ANDed with the inverted Off Duty switch (Figure 3.1). The Verilog implementation is shown in Listing 3.2.

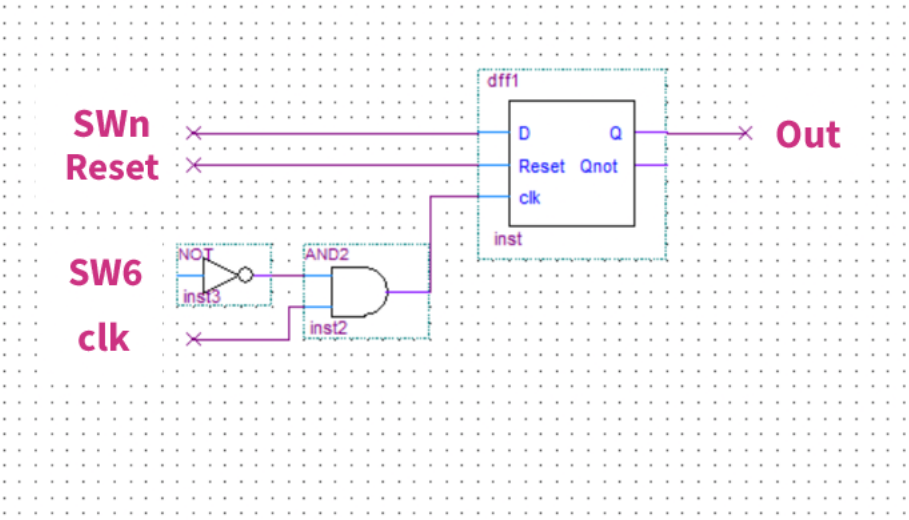


Figure 3.1: Block diagram of the input processing

```
module inputRegN (SW6, SW, zeroC, clk, Reset, Out);
input SW6, SW, zeroC, clk, Reset;
output Out;

Invgate (n6, SW6);
Andgate2 (w1, n6, clk);

//module mux2to1 (S, A, B, Out);
//module dff1(D,Reset,clk,Q,Qnot);

dff1 (SW, Reset, w1, Out, n);

endmodule
```

Listing 3.2: Verilog HDL implementation of the input processing

The next step is to check that each order switch was activated in the correct turn (order 1, order 2, …, order 5). For this, a second copy of the 3-bit counter was combined with a 3-to-5 decoder, so that the decoder output corresponding to the order whose turn it is will be ON. Each order switch is then ANDed with the decoder output that has its own number, as shown in the structural code of Listing 3.3.

```
module decoderwANDs(I2, I1, I0, S5, S4, S3, S2, S1, Y5, Y4, Y3, Y2, Y1);
input I2, I1, I0, S5, S4, S3, S2, S1;
output Y1, Y2, Y3, Y4, Y5;

Invgate (n2, I2);
Invgate (n1, I1);
Invgate (n0, I0);

Andgate3 (D4, I2, n1, n0);//Decoder D4 = I2.(~I1).(~I)
Andgate2 (Y5, D4, S5);// ANDed with the switch with resonating order

Andgate3 (D3, n2, I1, I0);//D3 = (~I2).I1.I0
Andgate2 (Y4, D3, S4);

Andgate3 (D2, n2, I1, n0);//D2 = (~I2).I1.(~I0)
Andgate2 (Y3, D2, S3);

Andgate3 (D1, n2, n1, I0);//D1 = (~I2).(~I1).I0
Andgate2 (Y2, D1, S2);

Andgate3 (D0, n2, n1, n0);//D0 = (~I2).(~I1).(~I0)
Andgate2 (Y1, D0, S1);

endmodule
```

Listing 3.3: Verilog HDL implementation of the order-turn check

The decoder inputs I2, I1 and I0 are the outputs of the 3-bit counter. For example, when the system starts, the counter is in state 000, so decoder output D0 is ON. This means that only SW1 is expected, and turning the other switches ON or OFF has no effect. To move on to the next order, the zero count (explained in Section 3.3.1) is used as the clock of this counter, since it indicates that the processing of the previous order is finished.

Figure 3.2 shows the complete input-processing block with all of its devices connected.

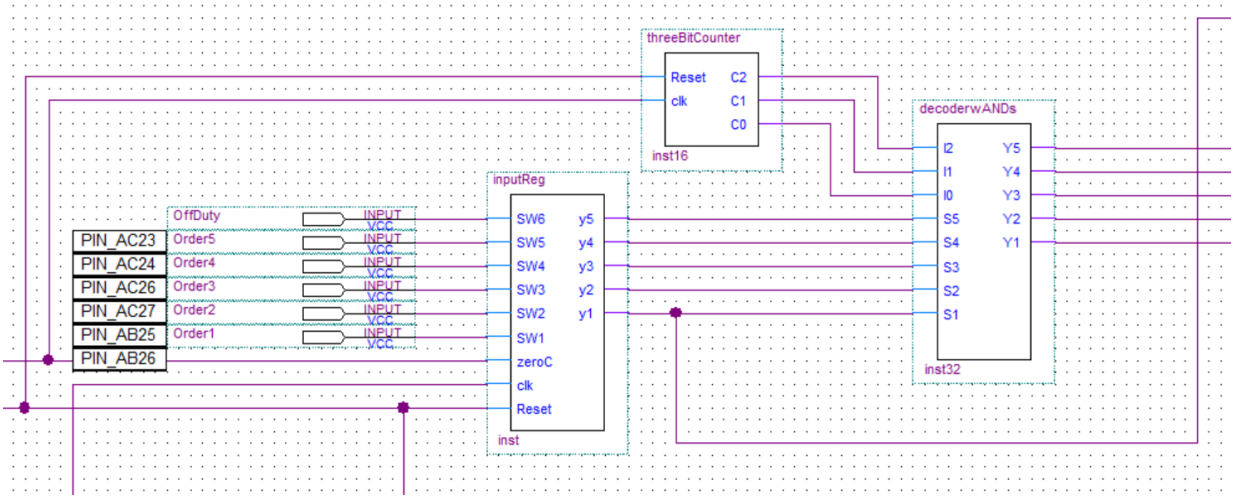


Figure 3.2: Connection of the devices that process the switch inputs

3.3 Orders Processing

3.3.1 Four-Bit Counter

When an order switch is turned ON in its correct turn, and both SW0 and SW6 are OFF, Display Digit 0 counts from 8 down to 0 as the delivery time. Since the largest number that a 3-bit register can hold is 7, a separate 4-bit counter was designed for this purpose.

The counter also provides two indicators that are used elsewhere in the design: an indicator for when the state reaches 0000, and a pre-zero indicator for state 0001 (described as the zero count in Lab Experiment 8), whose use is explained below. The states of the counter are given in Table 3.2.

Table 3.2: 4-bit counter states

Current State (C3 C2 C1 C0)	Next State (D3 D2 D1 D0)	Pre-Zero	Zero Count
0000	1000	0	1 (second zero)
0001	0000	1	0
0010	0001	0	0
0011	0010	0	0
0100	0011	0	0
0101	0100	0	0
0110	0101	0	0
0111	0110	0	0
1000	0111	0	0

The equations of the counter bits are:

```
D3 = (~C3)·(~C2)·(~C1)·(~C0)
D2 = C3 + C2·C1 + C2·C0
D1 = C3 + C1·C0 + C2·(~C1)·(~C0)
D0 = C3 + C1·(~C0) + C2·(~C0)
```

The counter must count from 8 to 0 and then stay at 0. This cannot be expressed in the state table alone, since the registers start at 0000. The counter was therefore designed to count normally, with its zero state leading back to 8. The pre-zero signal is set to 1 when the next state is 0000. One clock pulse later it sets the zero-count signal to 1, which marks the actual zero count, and combinational logic then holds it at 1. The zero count is used as the select line of a multiplexer that lets either the counter bit Dn or a constant 0 through to the output, so the outputs stay at zero once counting has finished (Figure 3.3). Listing 3.4 shows the Verilog implementation.

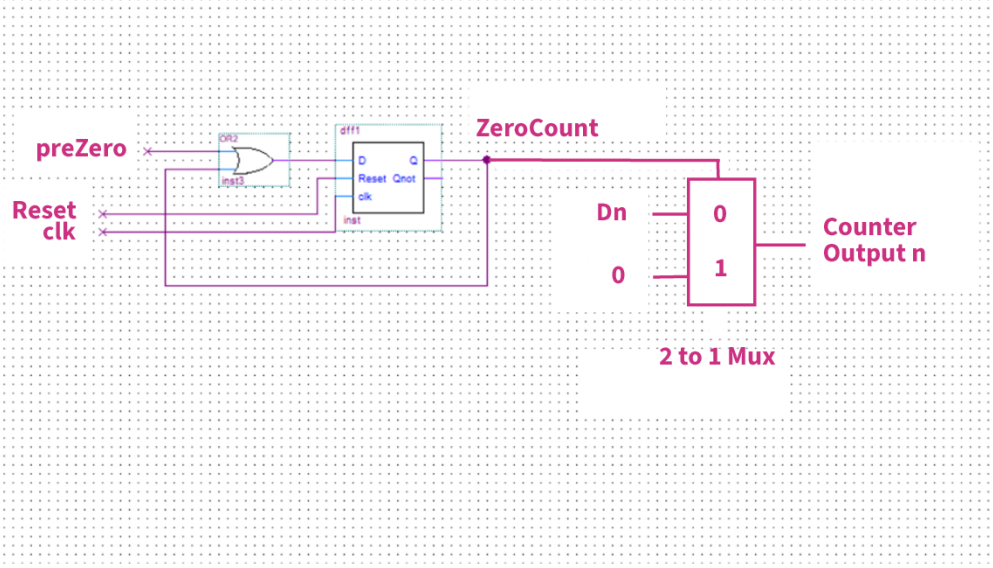


Figure 3.3: Block diagram of a single output bit of the 4-bit counter

```
module fourBitCounter (Reset, clk, C3, C2, C1, C0, preZero, zeroCount);
input Reset, clk;
output C3, C2, C1, C0, preZero, zeroCount;

Invgate (n0, C0);
Invgate (n1, C1);
Invgate (n2, C2);
Invgate (n3, C3);

Andgate4 (preZero, n3, n2, n1, C0);

//module dff1(D,Reset,clk,Q,Qnot);
Orgate2 (Q, preZero, zeroCount);
dff1 (Q, Reset, clk, zeroCount, nZero);

Andgate4 (B3, n3, n2, n1, n0);
//module mux2tol (S, A, B, Out);
mux2tol (zeroCount, 0, B3, B3z);

Andgate2 (w1, C2, C1);
Andgate2 (w2, C2, C0);
Orgate3 (B2, C3, w1, w2);
mux2tol (zeroCount, 0, B2, B2z);

Andgate2 (w3, C1, C0); //*
Andgate3 (w4, C2, n1, n0);
Orgate3 (B1, C3, w3, w4);
mux2tol (zeroCount, 0, B1, B1z);

Andgate2 (w5, C1, n0); //*
Andgate2 (w6, C2, n0); //*
Orgate3 (B0, C3, w5, w6);
mux2tol (zeroCount, 0, B0, B0z);


dff1 (B0z, Reset, clk, C0, nC0);
dff1 (B1z, Reset, clk, C1, nC1);
dff1 (B2z, Reset, clk, C2, nC2);
dff1 (B3z, Reset, clk, C3, nC3);

endmodule
```

Listing 3.4: Verilog HDL implementation of the 4-bit counter

This design creates a problem: the counter cannot be started again after the first order has been processed. To solve it, the restart input of the counter depends on more than the system reset switch. A second 3-bit order counter and 3-to-5 decoder (similar to the ones in Section 3.2) are used, with a different clock control.

The decoder outputs, each ANDed with its order switch, are combined in an OR gate that is connected to the clock of the counter. As soon as a switch is turned ON in its correct turn, the 4-bit counter restarts and the 3-bit counter moves to its next state. The OR output then returns to 0, which allows the 4-bit counter to start counting again.

The restart input must also be 1 before SW1 is turned ON (before any order is taken), to avoid conflicts with the 0-to-5 counter that is shown at that time. A D flip-flop with both its data input and its clock connected to SW1 stores the value 1 once deliveries begin and keeps it until the end of the operation (until the system reset switch is 1). Its inverted output is used as a restart for the 4-bit counter.

Finally, the OR gate of the decoder outputs, the inverted output of the SW1 flip-flop and the system reset switch are fed into a 3-input OR gate, whose output is connected to the restart pin of the 4-bit counter. The complete design of the counter is shown in Figure 3.4.

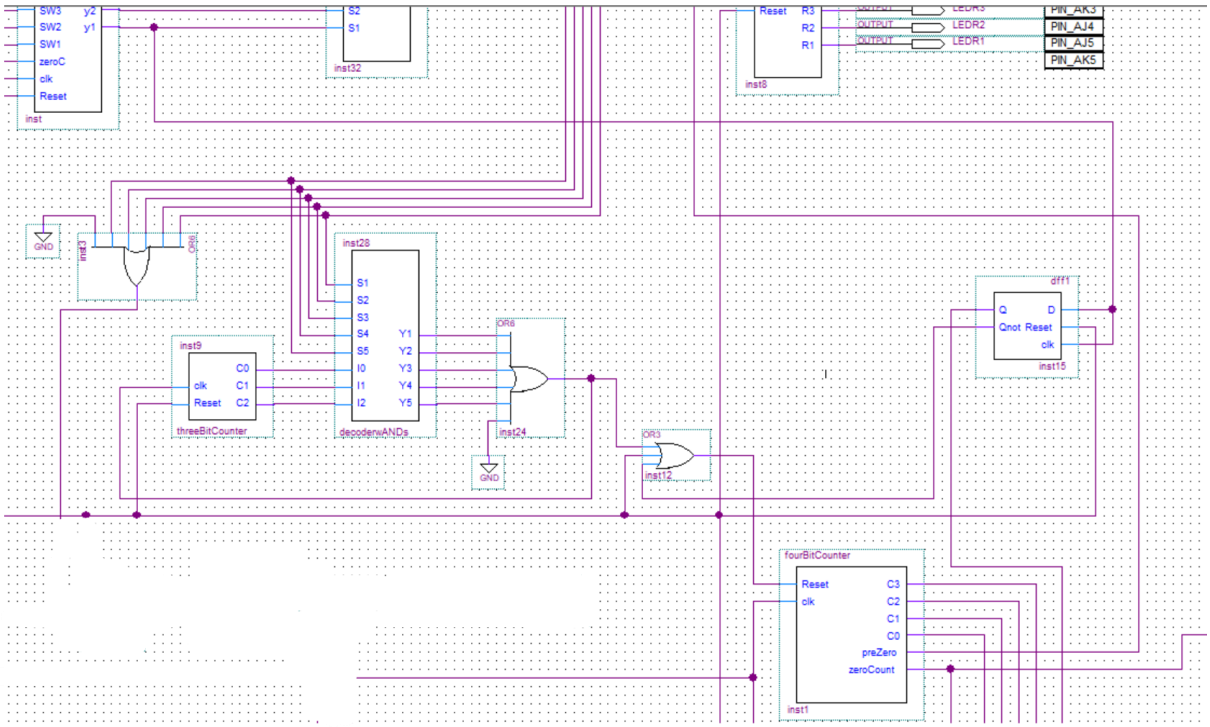


Figure 3.4: Connection of the 4-bit counter to the hardware that controls its restart

3.3.2 LED Device

The LEDs of each order are controlled by a demultiplexer, which routes the output to the green or red LED depending on whether the delivery succeeded. Two points had to be taken into account:

- The value of an order switch must be stored once it is turned ON, because the LED states stay until the end of the operation.
- The state of the delivery process (succeeded or failed) and the fact that the zero count has been reached must also be stored.

A D flip-flop stores the fact that a switch was turned ON in its correct turn. Its output is ANDed with the Success switch, and the result is saved in a second D flip-flop, whose output is the select line of the demultiplexer: it drives the green LED (G) and its inverse drives the red LED (R). Before reaching the

LEDs, the selection is ANDed (inside the demultiplexer) with the demultiplexer input, which is the output of the order-switch flip-flop ANDed with the zero count of that order. The output of this AND gate is also stored in a D flip-flop, so the values are kept until the end. When the zero count of the order is reached, the LEDs are therefore updated according to the result of the delivery: G = 1 and R = 0 if the Success switch was ON during the delivery time, and G = 0 and R = 1 otherwise.

The output of the flip-flop that stores the AND of the switch value and the zero count is also connected to the reset of the order-switch flip-flop. This prevents interference with the pins that are shared with other orders.

In addition, LED G must turn ON at the moment its order switch is turned ON in its correct turn, and then be updated when the zero count is reached. For this, a 2-to-1 multiplexer lets the value of the switch flip-flop through before the zero count of the order, and then lets the result of the delivery through at the zero state, as shown in Figure 3.5. The Verilog implementation is given in Listing 3.5.

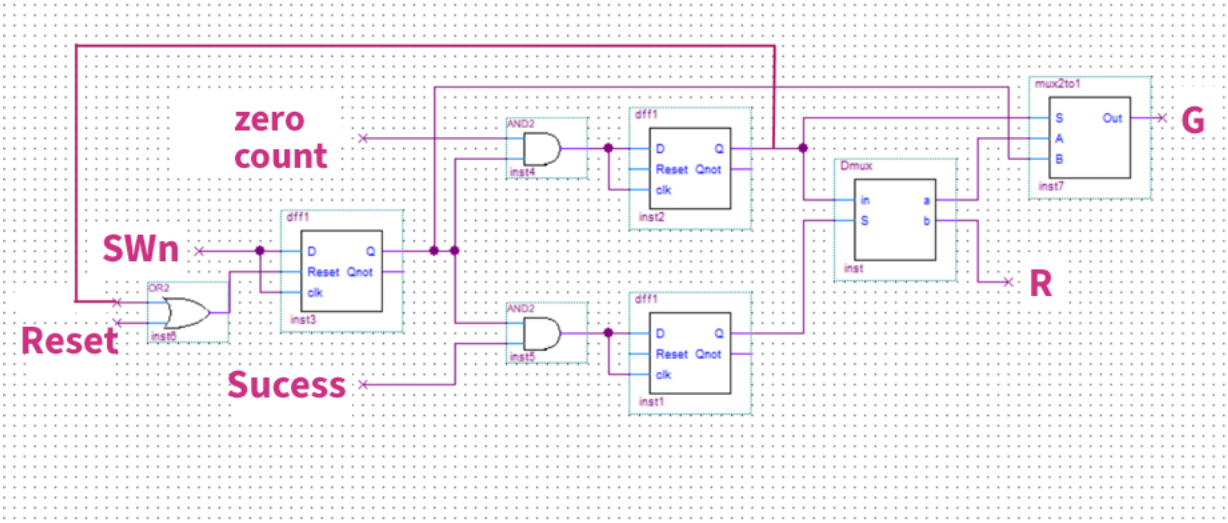


Figure 3.5: Block design of the device that processes the delivery results

```
module device (SW7, SW, zeroC, Reset, G, R);
input SW7, SW, zeroC, Reset;
output G, R;

dff1 (SW, Reset, SW, swReg, notSw);

Andgate2 (sel, swReg, SW7);
Andgate2 (in, swReg, zeroC);

//module dff1(D,Reset,clk,Q,Qnot);
dff1 (in, Reset, in, inReg, nIn);
dff1 (sel, Reset, inReg, selReg, nSel);//only updated at zero count of current order

//module Dmux (in, S, a, b);
Dmux (inReg, selReg, inG, R);

//module mux2to1 (S, A, B, Out);
mux2to1 (inReg, inG, swReg, G);


endmodule
```

Listing 3.5: Verilog HDL implementation of the LED device

3.3.3 Successful Deliveries Counter

Display Digit 1 shows the number of successful deliveries, which cannot exceed five. A copy of the 3-bit counter designed earlier is used for this purpose. Its clock is driven by an AND gate whose inputs are the OR of all the processed order inputs and an indicator that the Success switch was turned ON during the delivery time of the order. This indicator is a D flip-flop that stores the value of the switch once it is turned ON (its clock is connected to the switch input) and is reset at every zero count and on system reset, as shown in Figure 3.6.

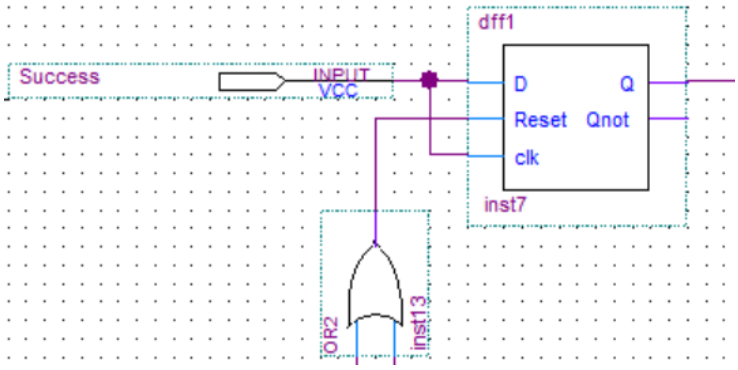


Figure 3.6: Processing of the Success switch to save its value in each round

The resulting counter design is shown in Figure 3.7.

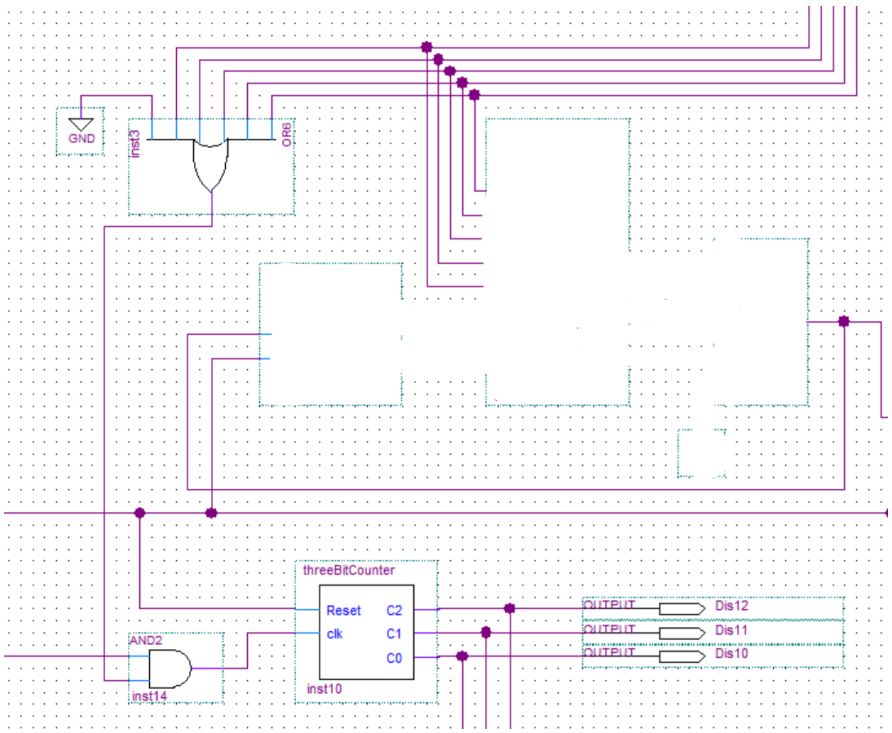


Figure 3.7: Block diagram of the 3-bit counter connections

3.3.4 Display Digits

To show numbers on the display digits, the counter outputs must be connected to a 7-segment decoder that translates them into the corresponding segment patterns. The code simply checks which segments must be ON for each number. Assuming common-anode displays, it is given in Listing 3.6.

```
module seg7 (A0, B0, C0, D0, a, b, c, d, e, f, g);
input A0, B0, C0, D0;
output a, b, c, d, e, f, g;

Invgate (nC, C0);
Xorgate (w1, B0, D0);
Andgate2 (a, nC, w1);

Xorgate (w2, C0, D0);
Andgate2 (b, B0, w2);

Invgate (nA, A0);
Invgate (nB, B0);
Invgate (nD, D0);
Andgate4 (c, nA, nB, C0, nD);

Andgate3 (w3, B0, C0, D0);
Orgate2 (d, w3, a);

Andgate2 (w4, B0, nC);
Orgate2 (e, D0, w4);

Orgate2 (w5, C0, D0);
Andgate2 (w6, nB, w5);
Andgate3 (w7, nA, C0, D0);
Orgate2 (f, w6, w7);

Andgate3 (w8, nA, nB, nC);
Andgate3 (w9, B0, C0, D0);
Orgate2 (g, w8, w9);

endmodule
```

Listing 3.6: Verilog HDL implementation of the 7-segment decoder

Display Digit 1 only shows the number of successful deliveries, so it is connected to a 7-segment decoder whose inputs are the outputs of the 3-bit counter that counts successful deliveries (Figure 3.8).

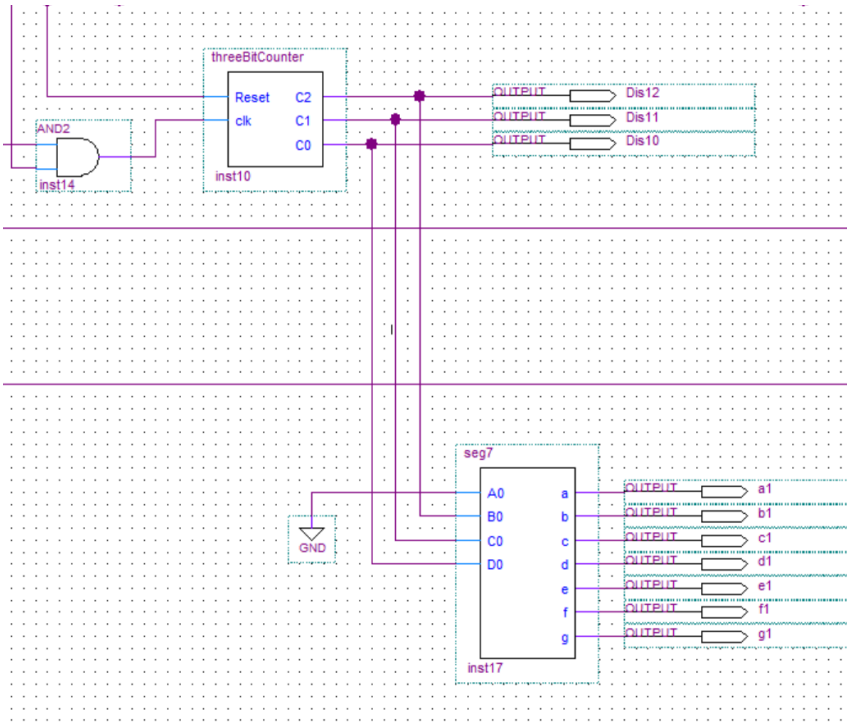


Figure 3.8: Display Digit 1 connected to its 7-segment decoder

Display Digit 0 has two modes. Before deliveries start, while the system is ON, it counts from 0 to 5. After order 1 starts, it shows the countdown from 8 to 0 for each order, which is the output of the 4-bit counter. Its 7-segment decoder is therefore connected to a 2-to-1 multiplexer that passes the output of the earlier 3-bit counter if deliveries have not started yet, and the output of the 4-bit counter otherwise. The select line of this multiplexer is the output of the D flip-flop described earlier, which is set to 1 when the first order starts (Figure 3.9).

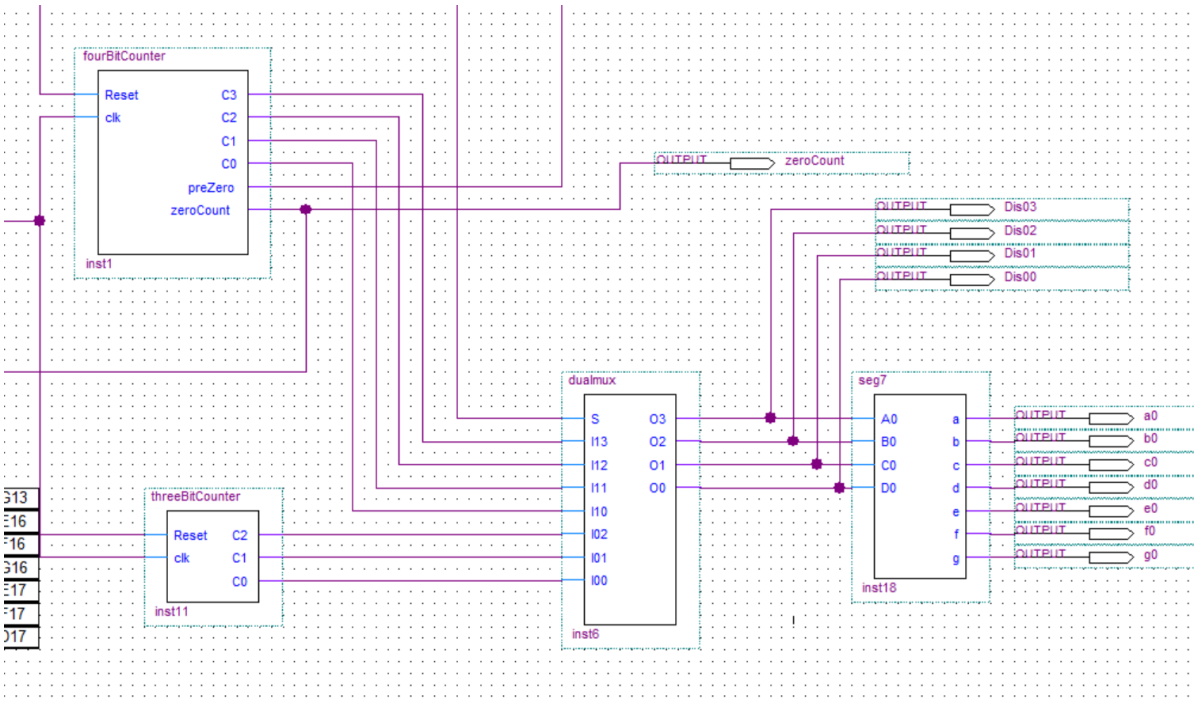


Figure 3.9: Display Digit 0 connected to its 7-segment decoder through a multiplexer

The Verilog HDL code of this multiplexer is shown in Listing 3.7. Note that in select mode 0 the most significant bit is set to 0, because the input is only 3 bits wide.

```
module dualmux (S, I13, I12, I11, I10, I02, I01, I00, O3, O2, O1, O0);
input S, I13, I12, I11, I10, I02, I01, I00;
output O3, O2, O1, O0;

//module mux2tol (S, A, B, Out);
mux2tol (S, I13, 0, O3);
mux2tol (S, I12, I02, O2);
mux2tol (S, I11, I01, O1);
mux2tol (S, I10, I00, O0);

endmodule
```

Listing 3.7: Verilog HDL implementation of the display multiplexer

4. Project Functional Simulation

Before presenting the simulation results, note that in the final design that is downloaded to the kit, the CLK input is connected to the provided clock device file. In simulation, however, it must be disconnected from this device so that the clock pulses required for each test case can be applied directly.

4.1 Simulation Case 1

Figure 4.1 shows the results of the first case. The following can be observed:

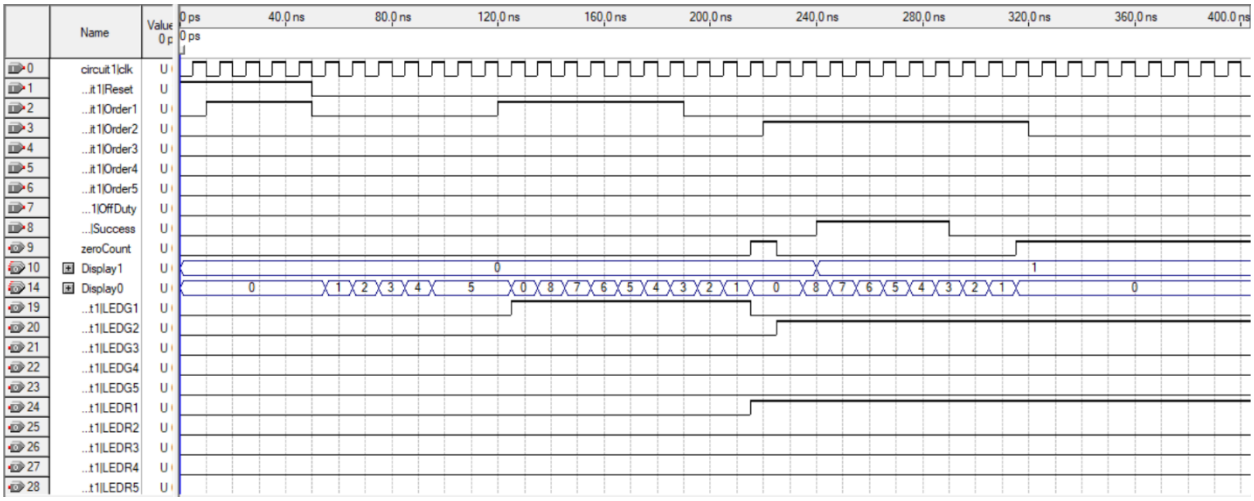


Figure 4.1: Simulation waveform of case 1

- While the Reset switch was ON, both display digits were set to 0, and turning switch 1 ON had no effect.
- After the Reset switch was turned OFF, Display Digit 0 started counting from 0 to 5.
- Activating switch 1 started order 1, so Display Digit 0 counted from 8 to 0 and the green LED of order 1 turned ON with the switch. Because the Success switch was not activated before the timer reached 0, the delivery was considered failed (a free pizza): the green LED turned OFF and the red LED turned ON.
- Order 2 was turned ON in its correct turn, so Display Digit 0 counted from 8 to 0 again. This time the Success switch was turned ON during the delivery time, so the green LED that had been ON since the beginning of the order stayed ON and the red LED stayed OFF.
- Display Digit 1 was incremented by 1 because of the successful delivery.
- When Display Digit 0 reached 0, it stayed at 0 as intended.

4.2 Simulation Case 2

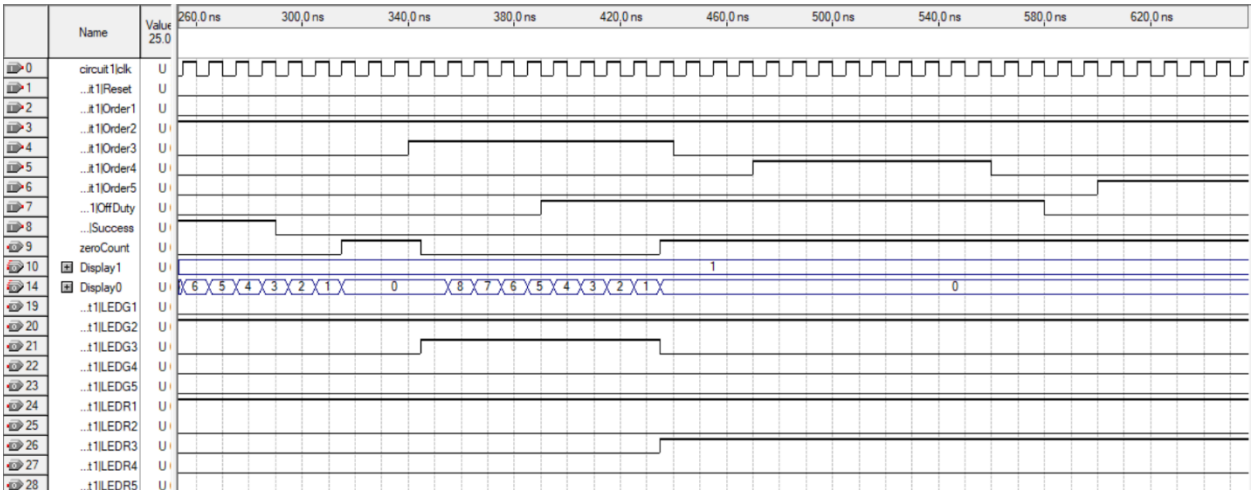


Figure 4.2: Simulation waveform of case 2

Observations for this case:

- Switch 2 remaining ON after its turn did not affect the other switches.
- The processing of order 3 continued normally, even though the Off Duty switch was turned ON during the delivery process.
- Since the Off Duty switch was ON, order 4 was not accepted. Consequently, after the Off Duty switch was turned OFF, switch 5 was still not accepted, because the correct sequence requires order 4 to be accepted first.

4.3 Simulation Case 3

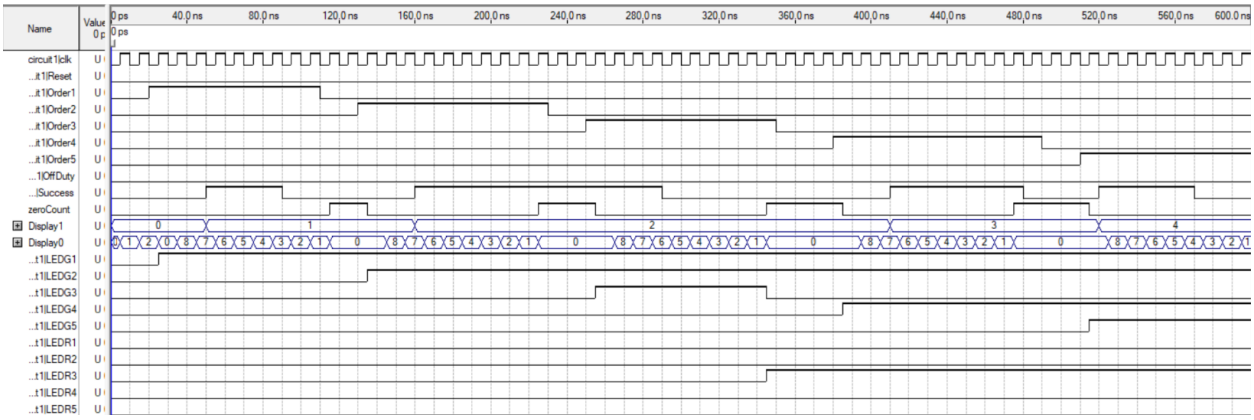


Figure 4.3: Simulation waveform of case 3

The key observation in case 3 is that all deliveries were successful except order 3. The Success switch staying ON from order 2 does not make order 3 successful; it must be turned OFF and then ON again. Display Digit 1 can also be seen to count the successful deliveries correctly.

4.4 Simulation Case 4

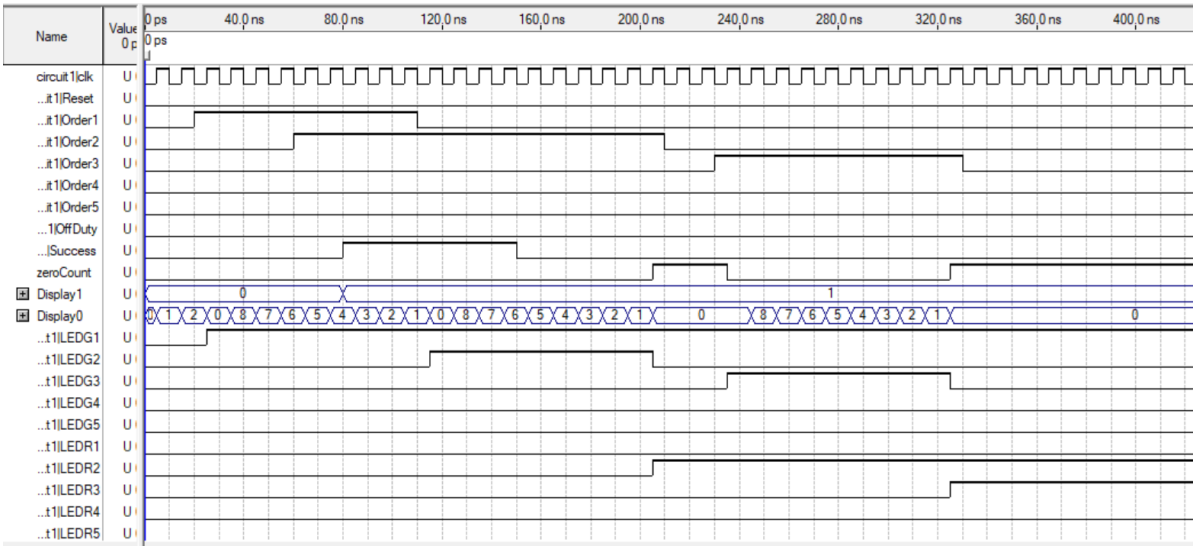


Figure 4.4: Simulation waveform of case 4

The key observation here is that turning switch 2 ON before the zero count of order 1 did not cause order 2 to be accepted. Order 2 was accepted as soon as order 1 was finished. Also, even though the Success switch was continuously ON during both orders, only order 1 was indicated as successful, as shown by the LEDs.

5. Conclusion

This project implemented a pizza delivery system that accepts up to five orders in sequence, counts down the delivery time of each order on a 7-segment display, indicates the outcome of each delivery with green and red LEDs, and counts the successful deliveries. The system was built from counters, decoders, multiplexers, a demultiplexer and D flip-flops, and was described structurally in Verilog HDL.

Functional simulation of four test cases confirmed the expected behavior: orders are accepted only in their correct sequence; the Off Duty switch allows the current order to finish while blocking new ones; a delivery counts as successful only if the Success switch is turned ON during its delivery time; and Display Digit 1 correctly counts the successful deliveries.